Übungsblatt 5 – Baumbasierte Verfahren & Neuronale Netze

Grundlagen Data Science & KI | DHBW

Aufgabe 1 – Grundbegriffe Decision Trees

(a) Erklären Sie die vier Bestandteile eines Decision Trees: Root Node, Splitter Nodes (innere Knoten), Internal Nodes und Leaf Nodes. Welche Aufgabe hat jede Knotenart?

(b) Was ist der Unterschied zwischen einem **Classification Tree** und einem **Regression Tree**? Welches Ergebnis liefert jeder Typ an den Blattknoten?

(c) Erklären Sie das Konzept des **Splitting**: Wie wird entschieden, welche Variable mit welchem Schwellenwert zum Teilen verwendet wird? Was wird dabei maximiert bzw. minimiert?

(d) Was sind **Surrogate Rules** und wofür werden sie bei Decision Trees benötigt?

(e) Nennen Sie je **zwei Vor- und Nachteile** von Decision Trees gegenüber linearen Modellen.

Aufgabe 2 – Decision Tree Szenario *(Berechnung)*

Ein Datensatz enthält 8 Datenpunkte, die beschreiben, ob Studierende eine Prüfung bestehen (**Ja**/**Nein**):

ID	Lernstunden	Schlaf gut?	Bestanden?
1	8	Ja	Ja
2	6	Ja	Ja
3	9	Nein	Ja
4	4	Ja	Nein
5	2	Nein	Nein
6	7	Nein	Ja
7	3	Ja	Nein
8	5	Nein	Nein

(a) Der Root Split wird auf **Lernstunden** ≥ 6 gesetzt. Teilen Sie den Datensatz anhand dieser Bedingung in zwei Gruppen auf und listen Sie jeweils die IDs und die Klasse der enthaltenen Datenpunkte.

(b) Bestimmen Sie für jede der beiden Gruppen das **Majority Voting**-Ergebnis. Wie lautet die vorhergesagte Klasse für jeden Blattknoten?

(c) Berechnen Sie für jeden Blattknoten die **Fehlerrate** (Anteil falsch klassifizierter Punkte).

(d) Eine neue Studentin hat 5 Lernstunden und schläft gut. Welche Klasse sagt der Baum voraus? Begründen Sie kurz.

Aufgabe 3 – Ensemble-Methoden

(a) Erklären Sie den Begriff **Ensemble Modeling**. Welches grundlegende Prinzip steckt dahinter?

(b) Beschreiben Sie das Verfahren **Bagging** (Bootstrapped Aggregation). Welche Fehlerkomponente wird dadurch primär reduziert? Begründen Sie.

(c) Beschreiben Sie das Verfahren **Boosting** (am Beispiel AdaBoost). Worin besteht der wichtigste Unterschied zu Bagging?

(d) Erklären Sie **Random Forest** als Kombination aus zwei Techniken. Was ist der Parameter **mtry**?

(e) Was ist ein **Out-of-Bag (OOB) Sample** und wofür wird es beim Bagging verwendet?

Aufgabe 4 – Neuronale Netze *(mit Berechnung)*

(a) Beschreiben Sie die biologische Analogie eines künstlichen Neurons: Welche biologischen Strukturen entsprechen Eingaben, Gewichten und der Aktivierungsfunktion?

(b) Ein einfaches Neuron erhält zwei binäre Eingaben x_1 und x_2. Die Gewichte betragen $w_1 = w_2 = 1$, der Schwellenwert beträgt $T = 1{,}5$. Das Neuron feuert, wenn $\sum_i w_i \cdot x_i \geq T$.

Berechnen Sie den Ausgang des Neurons für alle vier möglichen Eingabenkombinationen $(x_1, x_2) \in \{0, 1\}^2$. Welche logische Funktion modelliert das Neuron?

(c) Nun soll das gleiche Neuron (mit $w_1 = w_2 = 1$) das **XOR**-Problem lösen. Was ergibt sich für die Eingaben $(1, 0)$ und $(0, 1)$ bei $T = 1{,}5$? Kann ein einzelnes Neuron XOR lösen? Begründen Sie.

(d) Wie löst ein **Hidden Layer** das XOR-Problem? Beschreiben Sie den grundlegenden Mechanismus.

(e) Erklären Sie den Algorithmus **Backpropagation** in eigenen Worten: Welche Größen werden dabei angepasst und in welche Richtung wird die Fehlerinformation weitergegeben?

Aufgabe 5 – Bias-Variance-Trade-off Szenario

Drei Modelle (A, B, C) wurden auf demselben Datensatz trainiert. Die folgende Tabelle zeigt Trainings- und Testfehler:

Modell	Trainingsfehler	Testfehler
A	28 %	30 %
B	2 %	31 %
C	10 %	11 %

(a) Ordnen Sie jedem Modell eines der folgenden Phänomene zu: **Underfitting**, **Overfitting**, **guter Fit**. Begründen Sie jeweils anhand der Fehlerwerte.

(b) Welche Fehlerkomponente (Bias oder Varianz) ist jeweils dominant? Erklären Sie den Zusammenhang.

(c) Warum weist ein zu einfaches Modell hohen Bias auf, während ein zu komplexes Modell hohe Varianz aufweist? Erläutern Sie den grundlegenden Trade-off.

(d) Nennen Sie je eine konkrete Maßnahme, um Overfitting bei einem Decision Tree bzw. einem Neuronalen Netz zu reduzieren.

Aufgabe 6 – Vergleichstabelle: ML-Modelle

Füllen Sie die folgende Tabelle aus. Bewerten Sie jedes Kriterium mit + (gut/hoch), **o** (mittel) oder – (schlecht/gering) und geben Sie jeweils eine kurze Begründung:

Kriterium	**Decision Tree**	**Random Forest**	**Boosting**	**Neuronales Netz**
Interpretierbarkeit				
Umgang mit fehlenden Werten				
Overfitting-Risiko				
Trainingsaufwand				
Vorhersagegenauigkeit				
Nichtlineare Zusammenhänge				

Lösungen – Übungsblatt 5

Aufgabe 1 – Grundbegriffe Decision Trees

(a) **Root Node:** Der oberste Knoten des Baums, der die erste Entscheidung (Split) trifft. Er erhält alle Trainingsdaten als Eingabe. **Splitter Nodes / Internal Nodes:** Innere Knoten, die ebenfalls eine Bedingung (Variable + Schwellenwert) prüfen und die Datenpunkte in zwei Teilmengen aufteilen. **Leaf Nodes:** Blattknoten, die keine weiteren Splits durchführen und das Ergebnis liefern – bei Klassifikation eine Klasse, bei Regression einen Zahlenwert.

(b) Ein **Classification Tree** besitzt eine kategoriale Zielvariable (z. B. Ja/Nein). An jedem Blattknoten wird per **Majority Voting** die häufigste Klasse aller zugeordneten Trainingsbeispiele als Vorhersage ausgegeben. Ein **Regression Tree** besitzt eine kontinuierliche Zielvariable (z. B. Preis in Euro). An den Blattknoten wird der **Mittelwert** der enthaltenen Zielwerte zurückgegeben.

(c) Beim **Splitting** wird für jede mögliche Variable und jeden möglichen Schwellenwert berechnet, wie "rein" die resultierenden Teilmengen sind. Bei Klassifikation wird die **Unreinheit** (z. B. Gini-Impurity oder Entropie) **minimiert** – man sucht den Split, der möglichst homogene Gruppen erzeugt. Bei Regression wird die **Varianz** in den Teilmengen minimiert. Gewählt wird die Kombination aus Variable und Schwellenwert, die den größten **Information Gain** (Reinheitsgewinn) bringt.

(d) **Surrogate Rules** sind alternative Splitregeln, die für den Fall fehlender Werte (Missing Values) eingesetzt werden. Wenn ein Datenpunkt beim Primärsplit einen fehlenden Wert hat, wird stattdessen eine Surrogate-Variable genutzt, die dem Primärsplit möglichst ähnlich ist. Dies ermöglicht es dem Decision Tree, auch mit unvollständigen Daten korrekt zu klassifizieren, ohne Datenpunkte zu verwerfen.

(e) **Vorteile:** (1) Nichtlineare Entscheidungsgrenzen ohne manuelle Feature-Transformation möglich. (2) Direkt interpretierbar: der Baum kann als Regelwerk visualisiert werden (White-Box-Modell). **Nachteile:** (1) Keine Extrapolation möglich – der Baum kann nur innerhalb des Wertebereichs der Trainingsdaten vorhersagen. (2) Binäre Splits können zu Informationsverlust führen, wenn kontinuierliche Variablen mehrfach gesplittet werden müssen.

Aufgabe 2 – Decision Tree Szenario

(a) Split nach **Lernstunden** ≥ 6:

Gruppe Ja (≥ 6): IDs 1 (8h, Ja), 2 (6h, Ja), 3 (9h, Ja), 6 (7h, Ja) $\Rightarrow$ 4 Punkte, alle bestanden.

Gruppe Nein (< 6): IDs 4 (4h, Nein), 5 (2h, Nein), 7 (3h, Nein), 8 (5h, Nein) $\Rightarrow$ 4 Punkte, alle nicht bestanden.

(b) **Majority Voting:**

Linker Blattknoten (≥ 6): 4× Ja, 0× Nein $\Rightarrow$ Vorhersage: **Ja**.

Rechter Blattknoten (< 6): 0× Ja, 4× Nein $\Rightarrow$ Vorhersage: **Nein**.

(c) **Fehlerraten:**

Linker Blattknoten: 0 von 4 falsch $\Rightarrow$ Fehlerrate = $0/4 = 0{,}00 = 0\,\%$.

Rechter Blattknoten: 0 von 4 falsch $\Rightarrow$ Fehlerrate = $0/4 = 0{,}00 = 0\,\%$.

Dieser Split ist perfekt trennend (Fehlerrate gesamt = 0 %).

(d) Die neue Studentin hat 5 Lernstunden, also $5 < 6$. Der Baum leitet sie in den rechten Blattknoten. Dort lautet die Vorhersage **Nein (nicht bestanden)**.

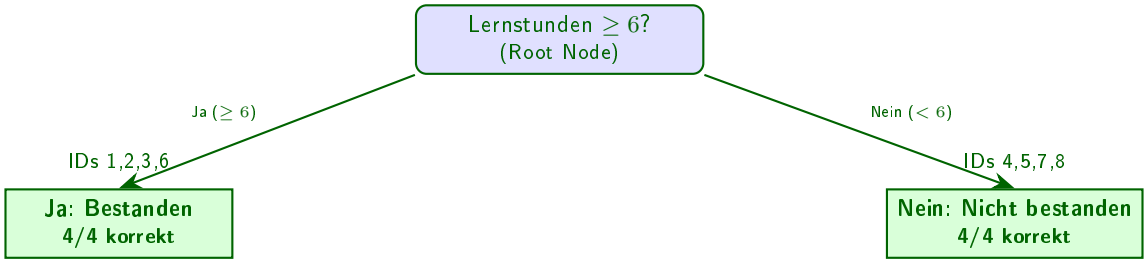

Abbildung: Decision Tree für den Prüfungs-Datensatz. Der einzige Split auf *Lernstunden* ≥ 6 erzielt 100 % Trainingsgenauigkeit – ein Hinweis auf mögliches Overfitting bei kleinen Datensätzen.

Aufgabe 3 – Ensemble-Methoden

(a) **Ensemble Modeling** ist ein Meta-Ansatz, bei dem mehrere Einzelmodelle kombiniert werden, um gemeinsam eine bessere Vorhersage zu erzielen als jedes Modell allein. Das Grundprinzip lautet: viele unabhängige schwache Lerner aggregiert ergeben einen starken Lerner. Die Unabhängigkeit der Modelle ist dabei entscheidend, damit sich Fehler nicht systematisch summieren.

(b) **Bagging** (Bootstrapped Aggregation) erstellt K zufällige Stichproben mit Zurücklegen (*Bootstrap Samples*) aus dem Trainingsdatensatz, trainiert auf jedem Sample einen unabhängigen Baum und aggregiert die Vorhersagen aller K Bäume (Mittelwert bei Regression, Mehrheitsvotum bei Klassifikation). Bagging reduziert primär die **Varianz** des Modells: hohe Varianz entsteht, wenn ein einzelner komplexer Baum stark auf die spezifischen Trainingsdaten reagiert – durch Mittelung vieler Bäume gleicht sich dies aus.

(c) **Boosting** (z. B. AdaBoost) trainiert die K Bäume **sequenziell**, nicht parallel. Nach jedem Baum werden die Gewichte der falsch klassifizierten Datenpunkte erhöht, sodass der nächste Baum sich stärker auf die schwierigen Fälle konzentriert. Der wichtigste Unterschied zu Bagging: die Bäume sind *abhängig* voneinander und das Verfahren reduziert primär den **Bias** (und teils auch die Varianz).

(d) **Random Forest** kombiniert (1) **Bagging** (Bootstrap Samples) und (2) **zufällige Feature-Auswahl** beim Splitting: an jedem Knoten wird nicht die beste Variable aus allen Features gewählt, sondern nur aus einer zufälligen Teilmenge von $m =$ mtry Features. **mtry** ist die Anzahl der Features, die an jedem Split zufällig in Betracht gezogen werden; typischerweise mtry $= \sqrt{p}$ bei Klassifikation ($p =$ Gesamtanzahl Features). Diese Randomisierung macht die Bäume unkorrelierter und reduziert die Varianz weiter.

(e) Das **Out-of-Bag (OOB) Sample** sind diejenigen Datenpunkte, die bei einem Bootstrap Sample *nicht* gezogen wurden (im Schnitt ca. 37 % des Datensatzes). Da sie nicht beim Training des jeweiligen Baums verwendet wurden, können sie als unabhängige Validierungsmenge dienen. Der OOB-Fehler dient damit als Schätzer für den Generalisierungsfehler – ohne separaten Testdatensatz oder Kreuzvalidierung.

Aufgabe 4 – Neuronale Netze

(a) Biologische Analogien: **Eingaben** x_i entsprechen den elektrochemischen Signalen an den **Dendriten**. Die **Gewichte** w_i entsprechen der Stärke der **synaptischen Verbindungen** – je stärker die Synapse, desto mehr Einfluss hat das Signal. Die **gewichtete Summe** entspricht der Integration aller Eingangssignale im Zellkörper (**Soma**). Die **Aktivierungsfunktion** modelliert das Schwellenwertverhalten des biologischen Neurons: überschreitet die Summe den Schwellenwert T, feuert das Neuron (**Aktionspotenzial** am Axon).

(b) Berechnung $\sum_i w_i \cdot x_i \geq T = 1{,}5$:

x_1	x_2	$w_1x_1 + w_2x_2$	$\geq 1{,}5$?	**Ausgang**
0	0	$0+0=0{,}0$	Nein	0
0	1	$0+1=1{,}0$	Nein	0
1	0	$1+0=1{,}0$	Nein	0
1	1	$1+1=2{,}0$	Ja	1

Das Neuron modelliert die logische **AND**-Funktion: es feuert genau dann, wenn beide Eingaben den Wert 1 haben.

(c) Für XOR mit $w_1 = w_2 = 1$, $T = 1{,}5$:

$(1,0)$: $1 \cdot 1 + 1 \cdot 0 = 1{,}0 < 1{,}5 \Rightarrow$ Ausgang = 0.

$(0,1)$: $1 \cdot 0 + 1 \cdot 1 = 1{,}0 < 1{,}5 \Rightarrow$ Ausgang = 0.

XOR erfordert aber Ausgang = 1 für $(1,0)$ und $(0,1)$. Ein einzelnes Neuron ist eine **lineare Klassifikation**: es trennt den zweidimensionalen Eingaberaum durch eine Gerade. XOR ist nicht linear trennbar – kein Schwellenwert T und keine Gewichte w_1, w_2 können XOR korrekt abbilden.

(d) Ein **Hidden Layer** ermöglicht nichtlineare Entscheidungsgrenzen. Die Hidden-Neuronen bilden zunächst neue (nichtlineare) Repräsentationen der Eingaben, die dann im Output-Neuron kombiniert werden. Für XOR kann ein Hidden Layer mit zwei Neuronen z. B. intern OR und NAND berechnen; deren Ausgaben ergeben erst kombiniert die XOR-Logik. Das Netz lernt damit eine *versteckte Schicht* von Zwischenrepräsentationen.

(e) **Backpropagation** berechnet den Fehler am Output und leitet ihn rückwärts durch das Netz (vom Output- zum Input-Layer). Dabei wird der Anteil jedes Neurons am Gesamtfehler (über die **Kettenregel** der Differentiation) bestimmt. Angepasst werden die **Gewichte** w_{ij} und die **Schwellenwerte** T_j jedes Neurons: sie werden in Richtung **negativen Gradienten** des Fehlers verschoben (Gradient Descent), sodass der Fehler bei der nächsten Iteration kleiner wird.

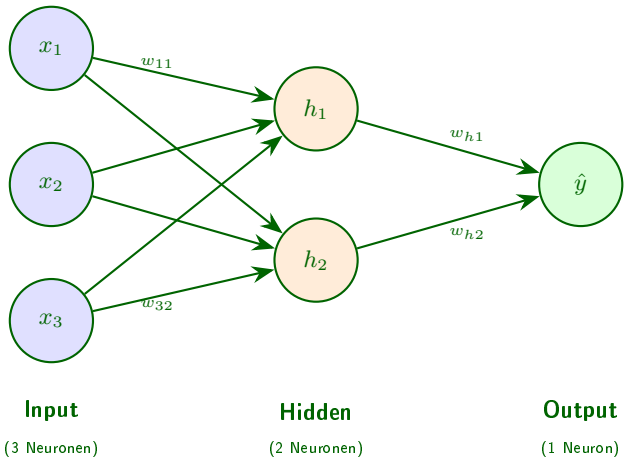

Abbildung: Neuronales Netz mit 3 Eingaben, einem Hidden Layer mit 2 Neuronen und einem Output-Neuron. Jede Verbindung trägt ein Gewicht w_{ij}. Das Hidden Layer ermöglicht nichtlineare Entscheidungsgrenzen.

Aufgabe 5 – Bias-Variance-Trade-off Szenario

(a) Zuordnung der Modelle:

Modell A – Underfitting: Trainings- und Testfehler sind beide hoch (28 % und 30 %) und ähnlich groß. Ein hoher Trainingsfehler deutet darauf hin, dass das Modell die Trainingsdaten bereits nicht korrekt abbilden kann – es ist zu einfach, um die Muster in den Daten zu erfassen.

Modell B – Overfitting: Der Trainingsfehler ist sehr gering (2 %), während der Testfehler sehr hoch ist (31 %). Diese große Lücke (*Generalisierungslücke* von 29 Prozentpunkten) zeigt, dass das Modell die Trainingsdaten nahezu auswendig gelernt hat, aber auf neuen Daten versagt.

Modell C – Guter Fit: Sowohl Trainings- (10 %) als auch Testfehler (11 %) sind moderat und sehr ähnlich. Die kleine Generalisierungslücke von 1 Prozentpunkt zeigt, dass das Modell echte Muster gelernt hat, ohne die Trainingsdaten auswendig zu lernen.

(b) **Fehlerkomponenten:** Modell A hat **hohen Bias**: das Modell ist zu einfach und macht systematische Fehler auf allen Datensätzen. Modell B hat **hohe Varianz**: das Modell reagiert zu stark auf die spezifischen Trainingsdaten und generalisiert nicht. Modell C zeigt einen guten Kompromiss mit **geringem Bias und geringer Varianz**.

(c) Ein zu **einfaches Modell** hat zu wenige Parameter, um die tatsächliche Struktur in den Daten abzubilden – es macht systematische Fehler, die auf allen Datensätzen gleich auftreten (*High Bias*). Ein zu **komplexes Modell** hingegen passt sich zu stark an die Zufälligkeiten der Trainingsdaten an (Rauschen, Ausreißer) und reagiert sehr empfindlich auf Schwankungen im Datensatz (*High Variance*). Es gibt keinen Weg, Bias und Varianz gleichzeitig auf Null zu reduzieren – man muss einen Kompromiss finden.

(d) **Decision Tree:** Pre-Pruning (maximale Tiefe begrenzen oder minimale Blattgröße festlegen) oder Post-Pruning (schwache Äste nach dem Training entfernen). **Neuronales Netz:** Dropout (zufälliges Deaktivieren von Neuronen während des Trainings) oder Early Stopping (Training beenden, wenn der Validierungsfehler zu steigen beginnt).

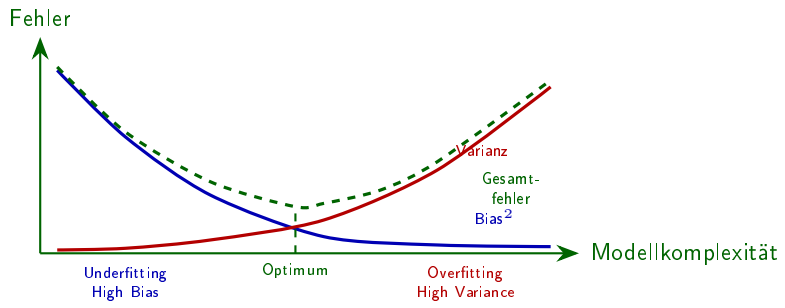

Abbildung: Bias-Variance Trade-off. Mit steigender Modellkomplexität sinkt der Bias, während die Varianz steigt. Der Gesamtfehler (gestrichelt) ist minimal am Optimum, wo beide Komponenten ausgewogen sind.

Aufgabe 6 – Vergleichstabelle: ML-Modelle

Kriterium	**Decision Tree**	**Random Forest**	**Boosting**	**Neuronales Netz**
Interpretierbarkeit	+ (visuell lesbar, Regeln ableitbar)	– (viele Bäume, Black Box)	– (sequenzielle Abhängigkeit)	– (Black Box, viele Parameter)
Fehlende Werte	+ (Surrogate Rules eingebaut)	+ (erbt vom Baum)	o (je nach Impl.)	– (Imputation nötig)
Overfitting-Risiko	– (hoch ohne Pruning)	o (Bagging reduziert es)	o (sensitiv auf Ausreißer)	– (hoch, Regularisierung nötig)
Trainingsaufwand	+ (schnell, ein Baum)	o (K Bäume parallel)	– (sequenziell, langsam)	– (viele Epochen, GPU nötig)
Vorhersagegenauigkeit	o (mittel)	+ (gut)	+ (oft bester Wert)	+ (sehr gut bei großen Daten)
Nichtlinear	+ (stückweise konstant)	+ (durch viele Bäume)	+ (durch Gewichtung)	+ (beliebig komplex)

Zusammenfassung: Der Decision Tree ist das einzige vollständig interpretierbare Modell in dieser Liste. Random Forest und Boosting bieten deutlich bessere Genauigkeit auf Kosten der Interpretierbarkeit. Neuronale Netze sind am flexibelsten und mächtigsten, erfordern aber die meisten Daten, den höchsten Rechenaufwand und die intensivste Regularisierung.